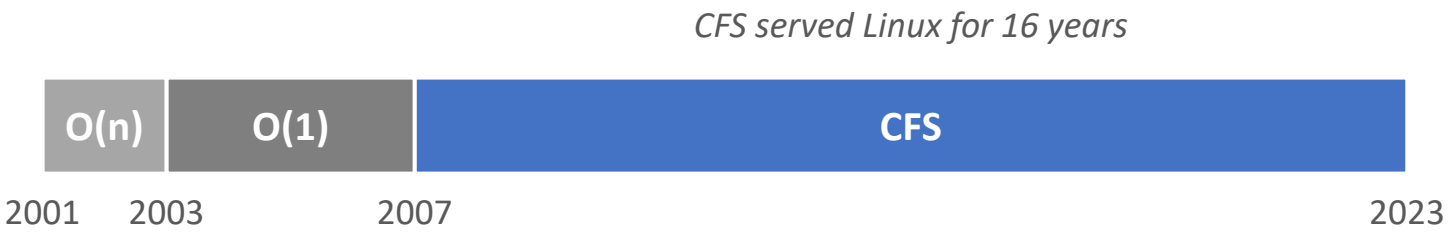
Linux EEVDF Scheduler

Earliest Eligible Virtual Deadline First



Linux Schedulers (for normal processes)

- O(n): Linux 2.4 to 2.6
- O(1): Linux 2.6 to 2.6.22
- CFS: Linux 2.6.23 to 6.5



Linux Schedulers (for normal processes)

- O(n): Linux 2.4 to 2.6
- O(1): Linux 2.6 to 2.6.22
- CFS: Linux 2.6.23 to 6.5
- **EEVDF: Linux 6.6 (Oct. 2023) to present**
  - Developed by Peter Zijlstra, replaces how CFS picks the next task
  - Keeps what we just learned: weights (nice), vruntime, red-black tree
  - Adds two new ideas: eligibility and virtual deadline



Motivation behind EEVDF

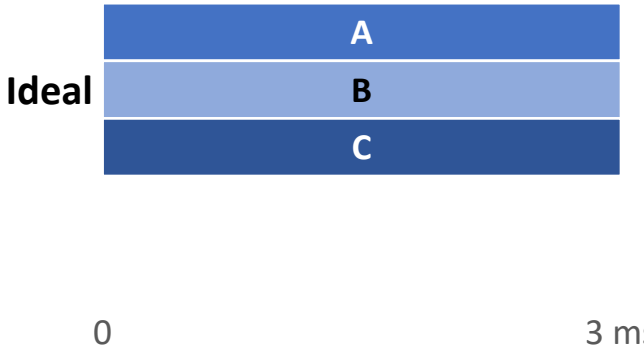
- Linux CFS is already a very good scheduler, yet
  - One knob (weight) decides both how much CPU and how soon a task runs
  - Latency is only a side effect: target latency, minimum granularity, wakeup granularity, sleeper credit
  - Many heuristics: hard to reason about and hard to tune
  - A task cannot say: “I need little CPU, but I need it soon”

Motivation behind EEVDF

- Linux CFS is already a very good scheduler, yet
  - One knob (weight) decides both how much CPU and how soon a task runs
  - Latency is only a side effect: target latency, minimum granularity, wakeup granularity, sleeper credit
  - Many heuristics: hard to reason about and hard to tune
  - A task cannot say: “I need little CPU, but I need it soon”
- Motivation for EEVDF: separate “how much” from “how soon”
  - Weight → share of CPU (**same as CFS**)
  - Request size (slice) → latency (**new**)
  - **Same RTSS 1996 paper by Ion Stoica et al. that we cited for CFS**
    - CFS used only half of it (virtual time); EEVDF uses the full algorithm

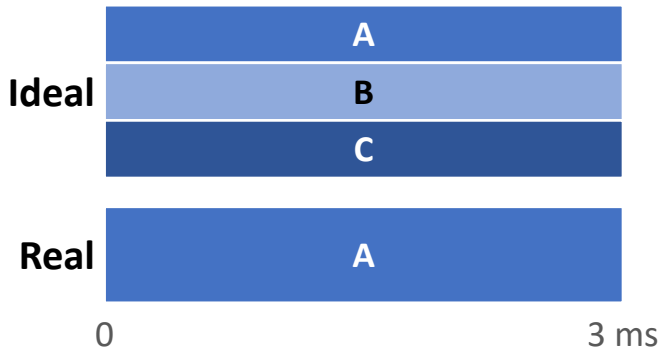
Back to the Ideal (Fluid) Scheduler

- Ideal scheduler: all runnable tasks run at the same time
  - Task i runs at rate w_i / W, where W = sum of all weights
  - In time t, task i should receive $t \times w_i / W$



Back to the Ideal (Fluid) Scheduler

- Ideal scheduler: all runnable tasks run at the same time
  - Task i runs at rate w_i / W, where W = sum of all weights
  - In time t, task i should receive $t \times w_i / W$
- Real CPU: one task at a time, so some tasks get ahead
- **Lag** = what a task should have got − what it actually got
  - lag > 0: behind (under-served); lag < 0: ahead (over-served)
  - The lags of all tasks always sum to 0



A, B, C have equal weights. After 3 ms:
Each should have received 1 ms
A received 3 ms → lag = 1 − 3 = **−2**
B, C received 0 ms → lag = 1 − 0 = **+1**
Sum of lags = −2 + 1 + 1 = 0

EEVDF Idea 1: Eligibility

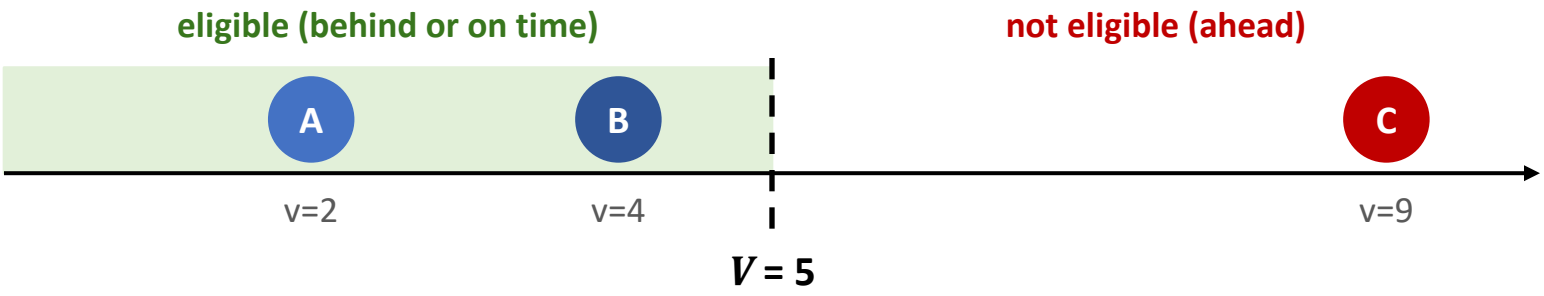
- A task is **eligible** only if lag ≥ 0
  - It has not received more than its fair share so far

EEVDF Idea 1: Eligibility

- A task is **eligible** only if lag ≥ 0
  - It has not received more than its fair share so far
- Linux computes this with vruntime (same vruntime as CFS)
  - V = weighted average vruntime = $\Sigma\, w_j\, v_j \,/\, \Sigma\, w_j$
  - $\text{lag}_i = w_i \times (V - v_i)$ → **eligible ⇔ $v_i \leq V$**

EEVDF Idea 1: Eligibility

- A task is **eligible** only if lag ≥ 0
  - It has not received more than its fair share so far
- Linux computes this with vruntime (same vruntime as CFS)
  - V = weighted average vruntime = $\Sigma\, w_j\, v_j \,/\, \Sigma\, w_j$
  - $\text{lag}_i = w_i \times (V - v_i)$ → **eligible ⇔ $v_i \leq V$**
- An over-served task must wait until the others catch up



Equal weights: V = (2 + 4 + 9) / 3 = 5. A and B may run; C must wait. (CFS would simply pick A.)

EEVDF Idea 2: Virtual Deadline

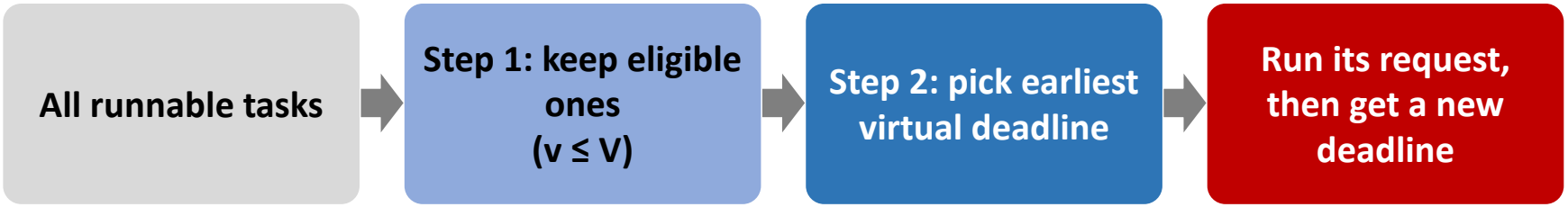
- Each task asks for a **request** of r_i ms of CPU (its **slice**)
- **Virtual deadline** = the virtual time at which this request would finish under the ideal scheduler
  - $vd_i = v_i + r_i / w_i$ (Linux: deadline = vruntime + slice × 1024 / weight)

EEVDF Idea 2: Virtual Deadline

- Each task asks for a **request** of r_i ms of CPU (its **slice**)
- **Virtual deadline** = the virtual time at which this request would finish under the ideal scheduler
  - $vd_i = v_i + r_i / w_i$ (Linux: deadline = vruntime + slice × 1024 / weight)
  - Shorter request → earlier deadline
  - Larger weight → earlier deadline (vruntime grows more slowly)
- After a task uses up its request, it gets a new deadline

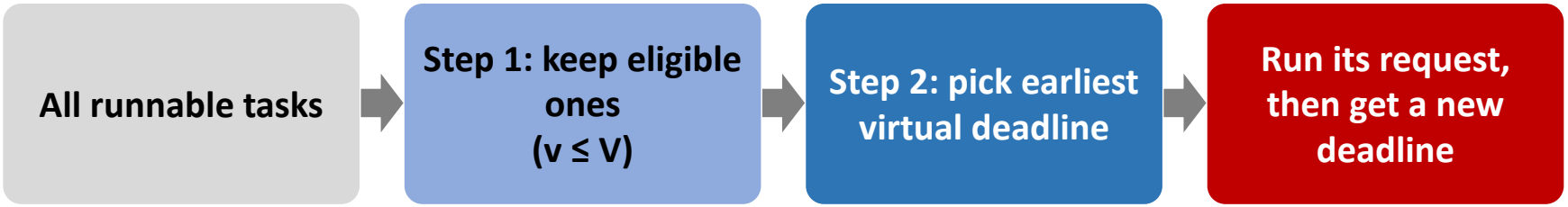
Task	Weight	Slice r	Virtual deadline
nice 0	1024	3 ms	v + 3 ms
nice 0	**1024**	**1 ms**	**v + 1 ms (earlier)**
nice −5	3121	3 ms	v + 3 × 1024/3121 ≈ v + 0.98 ms

The EEVDF Rule



- Eligibility gives **fairness**: no task runs ahead of its share
- Deadline gives **latency**: among fair candidates, the most urgent request goes first

The EEVDF Rule



- Eligibility gives **fairness**: no task runs ahead of its share
- Deadline gives **latency**: among fair candidates, the most urgent request goes first
- Efficiently tracked using a red-black tree
  - Sorted by virtual deadline (CFS: sorted by vruntime)
  - Each node also keeps the minimum vruntime of its subtree
  - Finding the earliest eligible task: O(log n)
- **CFS = "smallest vruntime first"; EEVDF = "earliest deadline among the eligible"**

EEVDF Step by Step

C: interactive, r = 1 ms. A, B, D: batch, r = 4 ms. All nice 0, always runnable.
A task runs its whole request, then EEVDF picks again.

Time	vruntime C / A / B / D	V	Eligible	Deadlines (eligible)	Run
0	0 / 0 / 0 / 0	0	C, A, B, D	C=1, A=4, B=4, D=4	**C**
1	1 / 0 / 0 / 0	0.25	**A, B, D**	A=4, B=4, D=4	**A**

At time 1, C has the earliest deadline (2), but C is **not eligible**: it already ran 1 ms while the others ran 0 ms (v = 1 > V = 0.25). So A runs.

EEVDF Step by Step

C: interactive, r = 1 ms. A, B, D: batch, r = 4 ms. All nice 0, always runnable.
A task runs its whole request, then EEVDF picks again.

Time	vruntime C / A / B / D	V	Eligible	Deadlines (eligible)	Run
0	0 / 0 / 0 / 0	0	C, A, B, D	C=1, A=4, B=4, D=4	**C**
1	1 / 0 / 0 / 0	0.25	**A, B, D**	A=4, B=4, D=4	**A**
5	1 / 4 / 0 / 0	1.25	C, B, D	C=2, B=4, D=4	**C**
6	2 / 4 / 0 / 0	1.5	B, D	B=4, D=4	**B**
10	2 / 4 / 4 / 0	2.5	C, D	C=3, D=4	**C**

At time 1, C has the earliest deadline (2), but C is **not eligible**: it already ran 1 ms while the others ran 0 ms (v = 1 > V = 0.25). So A runs.
Result: C runs every 5 ms, 1 ms at a time, and still gets exactly 1/4 of the CPU.

Request-Based Latency Control

Weight (nice) → how much

Long-term CPU share. Same as CFS.

Request (slice) → how soon

Short slice: runs soon and often, in short pieces. Long slice: fewer context switches.

- CFS: latency comes from one global target latency
  - Same for every task; gets worse as the number of tasks grows
- EEVDF: each task chooses its own request size
  - Short request → early deadlines → low latency, but the same CPU share

Request-Based Latency Control

Weight (nice) → how much

Long-term CPU share. Same as CFS.

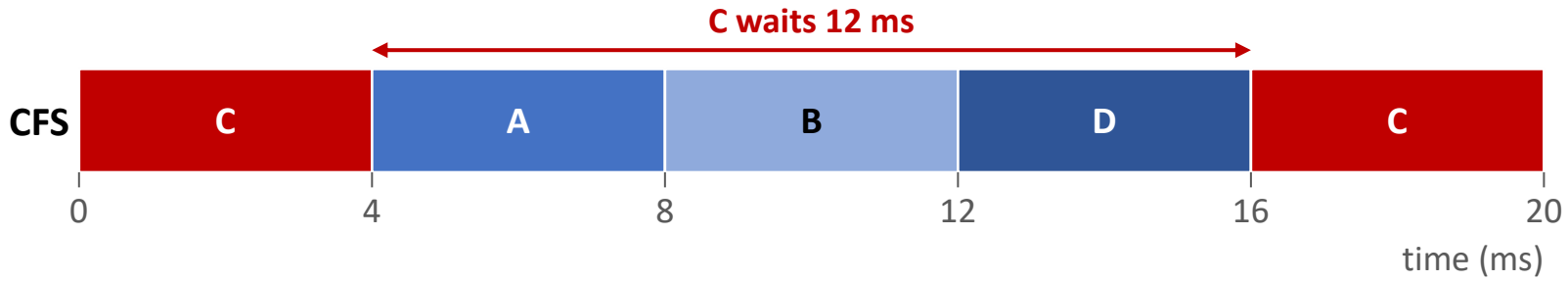
Request (slice) → how soon

Short slice: runs soon and often, in short pieces. Long slice: fewer context switches.

- CFS: latency comes from one global target latency
  - Same for every task; gets worse as the number of tasks grows
- EEVDF: each task chooses its own request size
  - Short request → early deadlines → low latency, but the same CPU share
- In Linux
  - Default slice: 0.75 ms × (1 + $\log_2$ #CPUs), e.g., 3 ms on 8 CPUs
  - Since Linux 6.12, a task can set its own slice (0.1 to 100 ms) with sched_setattr()
  - A waking task with a shorter slice can preempt the running task

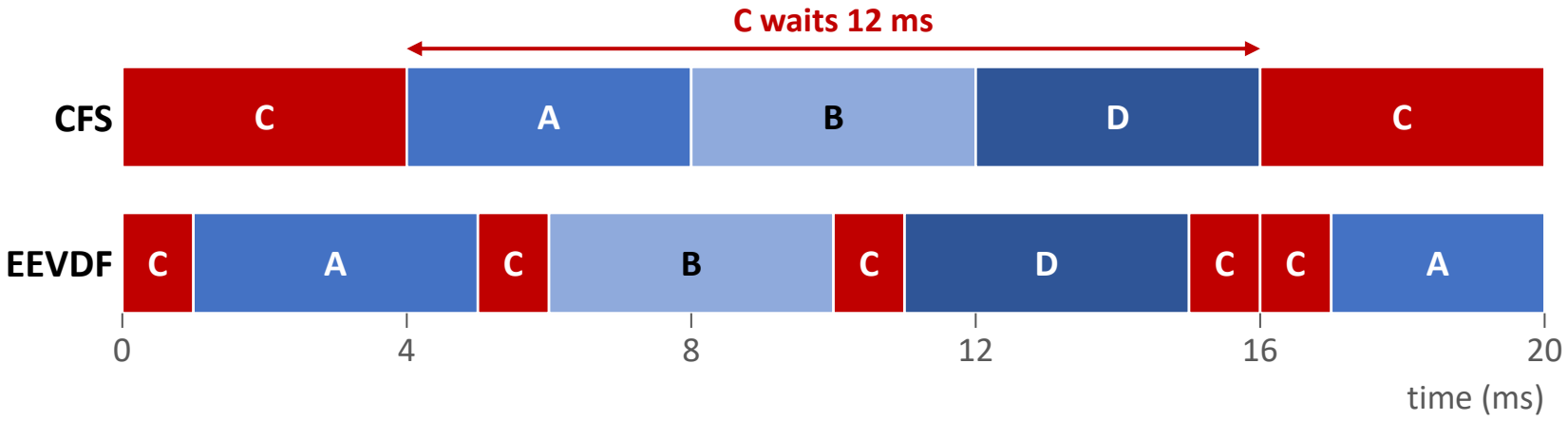
Example: CFS vs. EEVDF

- 4 tasks, all nice 0 → each should get 25% of the CPU
- A, B, D: batch (e.g., compiling). C: interactive (e.g., audio, UI), needs 1 ms at a time
- CFS: target latency 16 ms → 4 ms slices. EEVDF: A, B, D slice 4 ms, C slice 1 ms



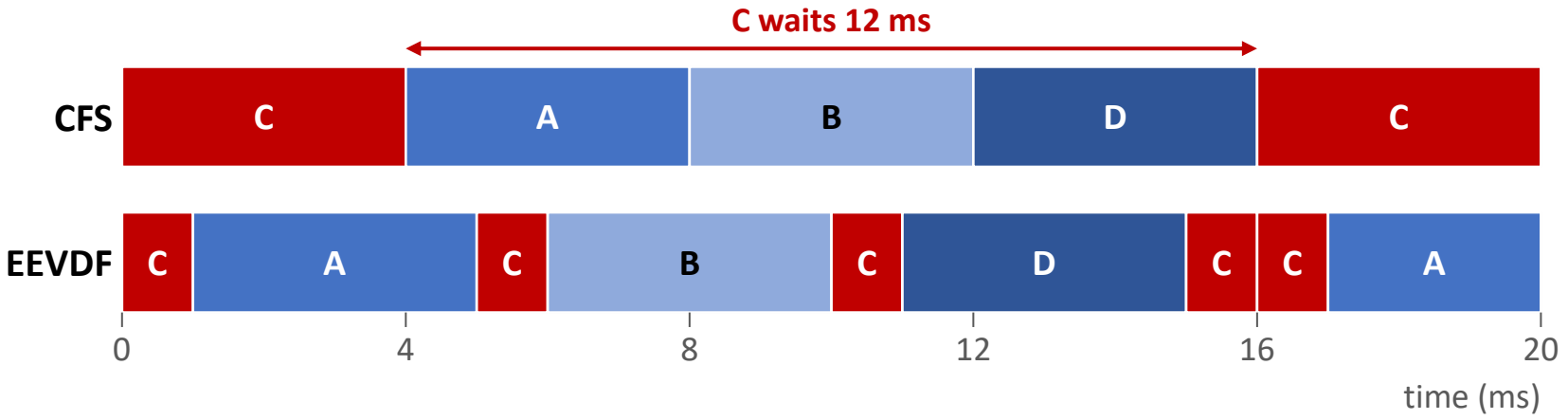
Example: CFS vs. EEVDF

- 4 tasks, all nice 0 → each should get 25% of the CPU
- A, B, D: batch (e.g., compiling). C: interactive (e.g., audio, UI), needs 1 ms at a time
- CFS: target latency 16 ms → 4 ms slices. EEVDF: A, B, D slice 4 ms, C slice 1 ms



Example: CFS vs. EEVDF

- 4 tasks, all nice 0 → each should get 25% of the CPU
- A, B, D: batch (e.g., compiling). C: interactive (e.g., audio, UI), needs 1 ms at a time
- CFS: target latency 16 ms → 4 ms slices. EEVDF: A, B, D slice 4 ms, C slice 1 ms



	CFS	EEVDF
CPU share of each task	25%	25%
Longest wait of C	**12 ms**	**4 ms**
Slice of A, B, D	4 ms	4 ms

Same fairness, 3× lower worst-case wait for C. With N batch tasks: CFS wait grows as 4N ms; EEVDF stays at 4 ms.

CFS vs. EEVDF

Concept	CFS	EEVDF
Pick rule	Smallest vruntime	Earliest virtual deadline among eligible tasks ($v \leq V$)
CPU share	Set by weight	Set by weight (same)
Latency	Side effect of target latency and number of tasks	Set per task by its request size (slice)

CFS vs. EEVDF

Concept	CFS	EEVDF
Pick rule	Smallest vruntime	Earliest virtual deadline among eligible tasks ($v \leq V$)
CPU share	Set by weight	Set by weight (same)
Latency	Side effect of target latency and number of tasks	Set per task by its request size (slice)
Waking tasks	Sleeper credit heuristic: vruntime set near min_vruntime − latency/2	Lag is remembered across sleep: no gain, no loss from sleeping
Tuning knobs	Target latency, minimum granularity, wakeup granularity	One base slice; each task may set its own
Guarantee	Approximately fair	Provable bound: lag never exceeds the largest request
Data structure	Red-black tree by vruntime	Red-black tree by deadline + min vruntime per subtree

When Is EEVDF Better?

- **Better**
  - Latency-sensitive tasks next to batch tasks: audio, video, UI, network handlers, robot control loops next to compilation or model training
  - Many runnable tasks: CFS latency grows with the number of tasks; EEVDF latency follows each task's request
  - Tasks that sleep and wake often: lag is kept, so no heuristics are needed

When Is EEVDF Better?

- **Better**
  - Latency-sensitive tasks next to batch tasks: audio, video, UI, network handlers, robot control loops next to compilation or model training
  - Many runnable tasks: CFS latency grows with the number of tasks; EEVDF latency follows each task's request
  - Tasks that sleep and wake often: lag is kept, so no heuristics are needed
- **About the same**
  - All tasks CPU-bound with the same slice: EEVDF behaves much like CFS
- **Possibly worse**
  - Very short slices → more context switches → lower throughput
  - Some workloads regressed after Linux 6.6; later kernels added fixes

Discussion Questions

Under EEVDF, task X has the earliest virtual deadline among all runnable tasks, but the scheduler does not pick it. What is the most likely reason?

A. X has the lowest weight.

B. X has received more than its fair share (its vruntime > V), so it is not eligible.

C. X requested a slice that is too short.

D. X has the largest positive lag.

Discussion Questions

Under EEVDF, task X has the earliest virtual deadline among all runnable tasks, but the scheduler does not pick it. What is the most likely reason?

A. X has the lowest weight.

B. X has received more than its fair share (its vruntime > V), so it is not eligible.

C. X requested a slice that is too short.

D. X has the largest positive lag.

Discussion Questions

Two tasks have the same nice value. Task A sets a 1 ms slice, and Task B uses a 4 ms slice. Both are always runnable. Which statement is true under EEVDF?

A. A gets 4× more CPU time than B.

B. Both get the same CPU share, but A runs more often in shorter pieces.

C. B gets 4× more CPU time than A.

D. A always preempts B immediately and B starves.

Discussion Questions

Two tasks have the same nice value. Task A sets a 1 ms slice, and Task B uses a 4 ms slice. Both are always runnable. Which statement is true under EEVDF?

A. A gets 4× more CPU time than B.

B. Both get the same CPU share, but A runs more often in shorter pieces.

C. B gets 4× more CPU time than A.

D. A always preempts B immediately and B starves.

Conclusion: CFS or EEVDF?

- On Linux 6.6 and later, EEVDF is the default; CFS's pick logic is gone
- EEVDF keeps the CFS fairness model
  - Weights, nice values, vruntime, red-black tree all carry over
- EEVDF adds what CFS lacked: a per-task latency knob with a clear guarantee

Conclusion: CFS or EEVDF?

- On Linux 6.6 and later, EEVDF is the default; CFS's pick logic is gone
- EEVDF keeps the CFS fairness model
  - Weights, nice values, vruntime, red-black tree all carry over
- EEVDF adds what CFS lacked: a per-task latency knob with a clear guarantee
- Which one to use?
  - Throughput only, all batch tasks: both work well; prefer longer slices
  - Latency-sensitive tasks mixed with batch tasks: EEVDF, and give the latency-sensitive tasks shorter slices
  - Hard deadlines: neither; use SCHED_DEADLINE or SCHED_FIFO

Choosing the Right Scheduler

I Care About:	Then Choose:
CPU Throughput	FCFS
Avg. Response Time	SRTF (preemptive SJF)
I/O Throughput	SRTF (preemptive SJF)
Fairness (Wait Time to Get CPU)	Round Robin
Fairness (Proportional)	Lottery, Stride, Linux CFS
Proportional Fairness + Per-Task Latency	**Linux EEVDF**
Meeting Deadlines	EDF
Favoring Important Tasks	Priority